

ICDFA LAB 2

File Upload & Command Execution Security Assessment

Professional Investigation Report

Assessment scope	DVWA + OWASP Mutillidae II in an isolated training environment
Assessment date	29 August 2026
Environment	Kali Linux • Docker • host-only/local vulnerable applications
Target addresses	DVWA: 10.5.5.12 Mutillidae II: 10.5.5.11
Testing constraints	Harmless markers only; no malware, reverse shell, persistence, privilege escalation, destructive commands, external callbacks, or unrelated-file access.

Executive conclusion: The DVWA assessment produced direct evidence of insecure file upload handling and OS command injection at the tested low security level. Mutillidae was restored to an operational state after its database initialization issue was resolved; however, the supplied session record does not contain completed Mutillidae upload and command-execution test results. Those items are therefore explicitly identified as pending rather than inferred.

1. Executive Summary

This assessment evaluated file-upload validation, storage handling, and command-execution trust boundaries in two intentionally vulnerable training applications. Testing was performed from Kali Linux against local Docker instances and used controlled, non-executable marker content.

DVWA findings. The evidence demonstrates that a valid 1×1 PNG marker was accepted and stored under the learner-supplied filename in the web upload directory. A content/filename mismatch was also accepted: a text marker was copied to a file named *lab2-mismatch.png*, and the server stored the file despite the content being ASCII text rather than PNG. The command feature accepted an IP address followed by a shell command separator and executed the appended harmless marker. The observed execution identity was **uid=33(www-data)**.

Mutillidae status. The application initially redirected to *database-offline.php*. Investigation showed MySQL running, the *mutillidae* database present, PHP MySQL extensions available, and direct PHP database selection succeeding. The application's own database check subsequently returned **AVAILABLE**, and the homepage returned HTTP 200. The remaining Mutillidae security-test evidence was not present in the supplied session record and is not fabricated in this report.

Overall, the strongest directly demonstrated weakness is command injection in DVWA at the tested low security level, because the modified input produced observable OS-command output while retaining the web-server execution context. Upload validation also showed a server-side content/extension trust weakness under the tested configuration.

Risk interpretation should distinguish observed impact from potential impact. No attempt was made to access credentials, unrelated files, persistence mechanisms, external systems, or privileged functions.

2. Scope, Rules and Method

The practical instructions required assessment of DVWA File Upload and Command Injection plus the equivalent Mutillidae challenges, comparison with stronger modes, evidence capture, remediation, and cleanup. The assessment was constrained to the assigned local/host-only vulnerable applications.

Testing methodology:

Phase	Method used / evidence
Preparation	Created text and PNG markers; recorded type, size, permissions and SHA-256.
Upload baseline	Submitted a harmless PNG to DVWA and captured successful storage behavior.
Mismatch	Submitted a text file named with a .png extension to test server-side content enforcement.
Storage	Inspected the Docker upload directory and verified stored type and permissions.
Command baseline	Submitted the normal DVWA ping input and recorded normal ICMP output.
Minimal execution proof	Appended a harmless echo marker to the IP input and observed marker output.
Context	Observed the executing identity as www-data.
Mutillidae readiness	Restored database and verified HTTP 200 plus application database availability.

3. Controlled Test Markers

The following learner-created files were used to minimize impact and make observations reproducible.

Marker	Type / size	SHA-256
lab2-marker.png	PNG, 1×1, 68 bytes	431ced6916a2a21a156e38701afe55bbd7f88969fbbfc56d7fe099d47f265460

Marker	Type / size	SHA-256
lab2-marker.txt	ASCII text, 32 bytes	8242651417ea05c42ec91ba992d06c7cfc99e1d3d06c56d0e38f7d88375af5d
lab2-mismatch.png	ASCII text, 32 bytes; deliberately named .png	8242651417ea05c42ec91ba992d06c7cfc99e1d3d06c56d0e38f7d88375af5d

The mismatch marker was verified locally with the `file` utility as ASCII text, then verified inside the DVWA container as ASCII text after upload.

4. DVWA Findings

4.1 File Upload — Normal Baseline

The DVWA upload challenge was tested at the low security level. The normal PNG marker was accepted with the application reporting: `../../hackable/uploads/lab2-marker.png successfully uploaded!`

Container verification showed the following stored files:

Stored file	Observed details
lab2-marker.png	68 bytes; PNG image data; owner/group www-data:www-data; mode -rw-r--r--
lab2-mismatch.png	32 bytes; ASCII text; owner/group www-data:www-data; mode -rw-r--r--

The normal upload request contained `filename="lab2-marker.png"` and `Content-Type: image/png`. This establishes the client request baseline.

4.2 Filename/Content Mismatch

A text marker was copied to `lab2-mismatch.png`. Local inspection established that the file was ASCII text, while its extension was `.png`. DVWA nevertheless reported successful upload. Container inspection subsequently confirmed that the stored `lab2-mismatch.png` remained ASCII text.

Conclusion: under the tested low-security configuration, the server did not enforce a content/extension consistency check strong enough to reject this harmless mismatch. This is evidence of weak server-side upload validation. It does not by itself prove executable-code upload or remote code execution.

4.3 Storage Handling

The files were stored in `/var/www/html/hackable/uploads/`. The supplied evidence shows learner-created files retained their submitted names and were placed under a web application directory. Stored permissions were non-executable for the files observed. The web-directory placement nevertheless warrants defense-in-depth controls because secure upload design should not depend solely on file permissions or filename extensions.

5. DVWA Command Injection

5.1 Baseline

The normal request was captured as a POST to `/vulnerabilities/exec/` with the body `ip=10.5.5.12&Submit;=Submit`. The response contained normal ping output for 10.5.5.12: four packets transmitted, four received, 0% loss.

5.2 Minimal Execution Proof

The modified request changed only the IP value to `10.5.5.12 && echo ICDFA-LAB2-CMD-MARKER`. The observed response contained the normal ping output followed by `ICDFA-LAB2-CMD-MARKER`. This is direct evidence that user-controlled input altered backend command structure rather than remaining pure data.

The observed execution context was `uid=33(www-data) gid=33(www-data) groups=33(www-data)`. This establishes the web-server account under which the demonstrated command executed.

5.3 Security interpretation

The minimum proof is sufficient because the test establishes the trust-boundary failure without requiring harmful payloads. The echo marker is deterministic, non-destructive, local, and directly attributable to the modified input.

6. Mutillidae II — Readiness and Evidence Status

The Mutillidae container was identified as *mutillidae.pc* with IP address 10.5.5.11. The application initially returned HTTP 302 to *database-offline.php*. Investigation established that MySQL was running, the *mutillidae* database existed, PHP MySQL extensions were installed, and direct PHP database selection succeeded.

The application's own database availability check returned **bool(true)** and **AVAILABLE**. A subsequent request to the root returned **HTTP/1.1 200 OK**, confirming that the application was restored to an operational state.

The available session record ends before completed Mutillidae upload and command-injection evidence was collected. Accordingly, this report does not claim findings that were not observed. The Mutillidae test rows in the evidence register are marked pending.

7. Observed vs Potential Impact

Area	Observed impact	Potential impact if exploitable more broadly
DVWA upload validation	Mismatched text content accepted as .png and stored.	Attackers may bypass weak type checks and place unexpected content in upload storage; impact depends on server execution/content-serving configuration.
DVWA upload storage	Learner files retained submitted names in a web application upload directory.	Predictable or web-accessible storage can increase exposure and, in poorly isolated deployments, contribute to content execution or unauthorized retrieval.
DVWA command injection	Appended echo command executed; output returned in HTTP response.	Command injection can permit arbitrary OS command execution with the web-server account and potentially lead to broader compromise if privileges or reachable resources allow it.
Execution context	www-data (UID/GID 33) observed.	Attacker capability would be bounded by the web account, but could still be serious depending on filesystem permissions, secrets, network access and application privileges.

8. Analysis Questions

1. Why is browser-only upload validation insufficient?

A browser control is under the user's control and can be bypassed by directly crafting HTTP requests. Validation that matters for security must occur server-side, independently of client-side JavaScript or form restrictions.

2. What proves command input was treated as executable structure rather than data?

The modified IP value contained a shell command separator and an echo instruction, and the response contained the resulting marker. The marker was not part of the intended ping target; its appearance demonstrates that backend command interpretation occurred.

3. Why is minimal proof sufficient?

The security claim concerns whether untrusted input crosses into an executable command context. A deterministic, harmless echo marker proves that boundary crossing without requiring destructive commands, credential access, persistence, or external callbacks.

4. Which control produced the greatest improvement in stronger mode?

A stronger-mode comparison was not completed in the supplied session record. Therefore no specific stronger-mode control is claimed as the greatest improvement. The correct conclusion after retesting should be based on which server-side control prevents the vulnerable input from changing command structure and which upload control rejects content/metadata mismatches.

9. Root-Cause Remediation

Weakness	Recommended layered controls	Root cause addressed
File upload validation	Allowlist extensions; validate MIME and magic bytes server-side; decode/re-encode images where appropriate; enforce size limits; reject mismatches; never trust client MIME headers.	Trust in attacker-controlled filename/content metadata.
Upload storage	Generate random server-side names; store outside executable web roots; disable script execution in upload directories; enforce least-privilege permissions; serve through controlled download endpoints.	Predictable names and unsafe storage/execution boundaries.
Command execution	Avoid shell invocation; use native language/library APIs; if an OS utility is unavoidable, use strict allowlists and structured argument APIs with no shell interpretation.	Concatenation of untrusted input into shell commands.
Execution context	Run application services with least privilege; isolate containers; minimize filesystem/network permissions; remove unnecessary credentials and capabilities.	Excessive blast radius after command execution.
Monitoring	Log rejected uploads, unexpected MIME/content mismatches, command errors and security events; alert on repeated validation failures.	Lack of visibility into attempted abuse.

10. Evidence Register

ID	Application	Evidence / action	Purpose
E01	Kali	lab2-marker.txt / lab2-marker.png properties	Controlled test content
E02	Kali	SHA-256 marker-sha256.txt	Integrity/repeatability
E03	DVWA	Normal PNG upload success	Baseline upload behavior
E04	DVWA	Normal upload POST request	Request evidence
E05	DVWA	lab2-marker.png in hackable/uploads	Storage behavior
E06	DVWA	lab2-mismatch.png accepted	Mismatch validation
E07	DVWA	Stored mismatch verified as ASCII text	Server-side validation conclusion
E08	DVWA	Normal command POST + ping response	Command baseline
E09	DVWA	Modified command + ICDFA marker	Minimal execution proof
E10	DVWA	uid=33(www-data)	Execution context
E11	Mutillidae	Database reset success	Application restoration
E12	Mutillidae	PHP DB selection OK / AVAILABLE	Database readiness
E13	Mutillidae	HTTP 200 homepage	Application readiness
E14+	Mutillidae	Upload/command tests	Pending in supplied session record

11. Activity Log Summary

29 Aug 2026 — Created `~/lab2-evidence/{markers,hashes,requests,responses,screenshots,notes}`. Created harmless text/PNG markers. Recorded file types, sizes and SHA-256 values. Tested DVWA upload and command injection at low security. Investigated Mutillidae database availability, reset the application database through its setup page, verified PHP/MySQL connectivity, and confirmed the application returned HTTP 200.

No malware, reverse shell, persistence, privilege escalation, destructive command, external callback, or unrelated-file access was performed.

12. Cleanup and Restoration

The lab instructions require removal of learner-created uploads/markers, resetting both applications, and verification that no listener, temporary account, scheduled task, or persistent artifact was created. The supplied session record does not contain final cleanup commands/results, so cleanup completion is marked **to be performed/verified** before submission.

Recommended final verification checklist:

- Delete only learner-created upload files from the two application upload locations.
- Reset both DVWA and Mutillidae to their expected training state.
- Verify no temporary listeners, accounts, scheduled tasks or persistent artifacts were created.
- Recompute/retain hashes of original learner markers if needed for evidence integrity.
- Archive the final report and evidence directory into the required ZIP package.

13. Final Assessment Conclusion

The directly observed DVWA results substantiate two security weaknesses under the tested configuration: insufficient server-side upload validation for a filename/content mismatch, and command injection in which attacker-controlled input was interpreted as shell command structure. The command-injection proof is particularly strong because the controlled marker appeared in the response and the execution identity was independently observed as www-data.

The Mutillidae instance was successfully restored and verified operational, but its remaining vulnerability tests were not completed in the supplied evidence. A professional report should never convert a planned test into a claimed finding; those rows remain pending until reproduced and evidenced.

Final submission recommendation: attach the raw Burp requests/responses, screenshots, terminal output, marker hashes, evidence register and activity log to the ZIP. Complete the Mutillidae tests and stronger-mode retest before submitting if the assessment rubric requires all objectives to be demonstrated.